U24AI001 - KARTHIK RAMKUMAR

LAB 10: DIFFIE–HELLMAN KEY EXCHANGE

1. Implementation and Sample Output

The program uses modular exponentiation by repeated squaring, random private values for Alice and Bob, public-key exchange, and independent shared-secret calculations. The sample output below is from the provided run; the random private keys and resulting values will change on subsequent executions.

```
============================================================
   Lab 10: Diffie-Hellman Key Exchange Implementation
============================================================

[+] Public Domain Parameters:
    Prime modulus (p) : 2147483647
    Generator     (g) : 7

[+] Step 1: Private Key Generation:
    Alice's Private Key (a) : 944519785
    Bob's Private Key   (b) : 936263435

[+] Step 2: Public Key Computation:
    Alice's Public Key (A = g^a mod p) : 330525735
    Bob's Public Key   (B = g^b mod p) : 1970785332

[+] Step 3: Public Key Exchange:
    Alice sends public key A (330525735) to Bob.
    Bob sends public key B (1970785332) to Alice.

[+] Step 4: Shared Secret Calculation:
    Alice computes: K_Alice = (B^a mod p) = 1584415
    Bob computes  : K_Bob   = (A^b mod p) = 1584415

[+] Step 5: Verification:
    [SUCCESS] Both shared secret keys match!
    Established Shared Key: 1584415
============================================================
```

3. Algorithm

• Choose public parameters: a prime modulus p and a generator g.

• Alice selects private value a and computes A = g^a mod p.

• Bob selects private value b and computes B = g^b mod p.

• Alice and Bob exchange the public values A and B.

• Alice computes K_A = B^a mod p; Bob computes K_B = A^b mod p.

• Both values equal g^(ab) mod p, so the shared keys should match.

4. Short Report Answer

An eavesdropper can observe the public **parameters p and g,** as well as the **public values A and B**, but **does not know Alice's private value a or Bob's private value b**. Recovering a private value from a public value requires solving a **discrete logarithm problem**, which is **computationally difficult** when secure, sufficiently large parameters are used. Without a private value, the attacker **cannot straightforwardly calculate** the shared **secret g^(ab) mod p.** Both participants **can independently calculate the same secret** using their own private value and the other party's public value.

5. Discussion Questions

Q1. Why can't an attacker easily compute the shared key?

The attacker sees p, g, A, and B, but not the private values a and b. Computing the shared key from these public values is designed to be computationally infeasible when appropriately large and secure parameters are used.

Q2. Which mathematical problem must the attacker solve?

The attacker would need to solve the discrete logarithm problem to recover a private exponent from a public value, for example finding a from A = g^a mod p.

Q3. Why do Alice and Bob obtain the same shared key?

Alice calculates (g^b)^a mod p and Bob calculates (g^a)^b mod p. Both expressions equal g^(ab) mod p because ab = ba, so the resulting shared keys are equal.

6. Compilation and Execution

Compile using C++17:

```
g++ -std=c++17 dh.cpp -o dh
```

Run the program:

```
./dh
```

Submission reminder: place this report PDF, dh.cpp, and output.txt (containing one sample run) in the same folder before creating the ZIP archive, as required by the lab handout.